

IEEE SCS EGYPT CHAPTER 2026
STUDENT DESIGN COMPETITION

Design, Verification, and Implementation of a Resource-Efficient Streaming CNN Convolution Accelerator on FPGA

FPGA-Based Edge-AI Vision Accelerator

PROJECT

Target Board PYNQ-Z2
Target FPGA Xilinx Zynq-7020
Device xc7z020c1g400-1
Toolchain Vivado, ModelSim, Python 3
Repository [GitHub Source Code](#)

TEAM

Team FlipFlopers

Members:

Ahmed Badwy
Ahmed Amir
BadrEldin Hossam

University Cairo University

FlipFlopers

Design • Verification • FPGA Implementation

September 2026

Contents

1	Introduction	1
2	Competition Requirements and Design Targets	2
3	System Architecture	2
4	Parallel Multi-Kernel Extension (Bonus Feature)	3
5	Streaming Data Path	4
6	Window Generation and Line Buffers	5
6.1	RTL Implementation (<code>window_nxn</code>)	6
7	Kernel Storage and Programming	7
8	Pipelined MAC / Convolution Engine	7
9	Control FSM and Streaming Protocol	8
10	Fixed-Point Arithmetic and Bit-Width Analysis	9
10.1	Width Derivation	9
10.2	ReLU Behavior (Bonus Feature)	9
11	Verification Methodology	9
12	Python Golden Model	10
13	Testbench Architecture	10
14	Verification Results	11
15	System-Level Integration (Bonus Feature)	12
15.1	Wrapper Architecture	12
15.2	ModelSim System Verification	12
15.3	Why the Wrapper Is Kept Lightweight	13
16	Edge-Detection Demonstration (Bonus Feature)	13
17	Design-Space Exploration	15
17.1	Multiplier Architecture Exploration	15
17.2	Why the DSP Cascade Wins	17
17.3	SRL versus FF Storage and Reset Policy	18
17.4	Frequency versus FOM	18
18	FPGA Implementation	18
18.1	Asymmetric Constraining Strategy	19
19	Resource Utilization	21
20	Timing and Maximum Frequency	21
21	Power Analysis and Post-Implementation Verification	22
21.1	Architectural Power Optimizations	22
21.2	Post-Implementation Timing and Functional Simulation	23

21.3 Active Window Isolation via SAIF Profiling.....	23
21.4 Voltage Scaling Optimization	24
22 Throughput and Latency	25
23 Figure of Merit	25
24 Discussion	26
24.1 Key Findings	26
24.2 Per-Cycle FOM and Resource Minimization.....	26
24.3 Verification Depth.....	27
24.4 Synthesis Sensitivity at Small Design Sizes.....	27
24.5 Remaining Headroom	27
25 Reproducibility and Tool-Version Considerations	27
26 Conclusion	28
References	28
A Competition Requirements Matrix	28
B RTL File Organization	30
C Verification Configuration Matrix.....	30
D RTL Module Summary	30

Abstract

This report presents the design, verification, and measured implementation of a 3×3 streaming CNN convolution accelerator developed for edge-AI inference on a Xilinx Zynq-7020 FPGA. The architecture combines a run-time programmable signed 8-bit kernel bank, an $N\times N$ sliding-window generator built from row line buffers, a triangular tap-delay array, and N^2 DSP48E1 multiply-accumulate units chained through the DSP cascade, producing one full-precision output pixel per clock cycle at a 400 MHz (MAX FREQ) clock-summary point. A layered verification stack – an independent Python golden model with generated expected-output files, a structural RTL mirror, a closed-form output law check, exact per-burst strobe accounting, and protocol assertion guards – validates the baseline bit-exactly and supports a wider parameterized regression space. On the current 3×32 baseline configuration, with $K=1$ and ReLU enabled, the routed design occupies **51 LUTs**, **97 flip-flops**, **9 DSP48E1 slices**, and **zero BRAMs**. Through active SAIF window profiling and voltage scaling, the post-implementation total power estimate was optimized to **0.124 W** and a routed timing result is WNS +0.005 ns at a 2.5 ns clock period, corresponding to a measured clock summary frequency of **400 MHz (MAX FREQ)**. With the competition throughput definition of one output pixel per cycle, the updated FOM is **0.01610**. Beyond the final result, the report documents a measurement-driven design-space exploration of alternative multiplier architectures, reset policies, accumulator gating, and parameter scaling. The multiplier study progressed from fabric implementations to a measured DSP-free reference (792 LUTs, 0 DSPs, 143 MHz MAX FREQ, FOM 0.01010 measured at its own MAX-FREQ operating point) and ultimately established the DSP48E1 mapping as the preferred measured implementation for the competition metric. A lightweight bonus system-level wrapper was also verified in ModelSim, integrating kernel loading, image buffering, start/busy/done control, and the existing streaming CNN core without an AXI interface.

Keywords – FPGA, CNN accelerator, edge AI, DSP48E1, streaming architecture, line buffers, fixed-point arithmetic, bit-exact verification, figure of merit, voltage scaling.

1 Introduction

Convolution is the dominant arithmetic primitive of convolutional neural networks and typically accounts for the majority of both compute and memory traffic in edge inference. For low-cost edge deployment, FPGAs offer deterministic timing, fine-grained parallelism, and direct control over the arithmetic datapath – advantages that are difficult to obtain from fixed-ISA processors. A streaming architecture, in which pixels flow through the datapath and one result emerges per clock cycle in steady state, naturally matches the sensor-to-output dataflow of embedded vision systems and avoids storing entire frames on chip.

This work was developed for the IEEE SSCS Egypt Chapter 2026 Student Design Competition. The competition scores submissions by a single figure of merit (FOM),

$$\text{FOM} = \frac{\text{Throughput}}{P \times (\text{LUT} + 50 \cdot \text{DSP} + 100 \cdot \text{BRAM})}, \quad (1)$$

where throughput is defined in *output pixels per clock cycle* and P is the total on-chip power in watts. Equation (1) shaped the architecture in three ways. First, it prices a DSP48 slice at only 50 points while a fabric multiply-accumulate costs 75–110 LUTs, strongly favoring DSP-based multiplication. Second, it prices a BRAM tile at 100 points, excluding block memories for the small on-chip storage this design needs. Third, because throughput is measured in *output pixels per cycle*, the clock frequency does not appear explicitly in the FOM equation. However, frequency affects the implementation’s dynamic power, which is part of the FOM denominator. Therefore, every reported FOM value is tied to the power estimate at the stated operating frequency. Unless otherwise stated, the implementation metrics reported in this document – resource utilization, power, timing, and the derived FOM – refer to the same routed $N=3$, $W=32$, $K=1$ implementation at the selected maximum-frequency operating point of 2.5 ns (400 MHz (MAX FREQ)). The

400 MHz (MAX_FREQ) result is therefore reported as the achieved maximum-frequency operating point, while the competition FOM remains determined by throughput per cycle, power, and counted resources.

Throughout this report, claims are labeled by their evidence class: *measured* (routed implementation or simulation output), *derived* (computed from measured values by a stated formula), or *analytical* (bounded by construction). Values that could not be established are marked TODO rather than estimated.

2 Competition Requirements and Design Targets

The IEEE SSCS Egypt 2026 Student Design Competition mandates the development of a resource-efficient streaming CNN convolution accelerator targeting edge-AI inference on a Xilinx Zynq-7020 FPGA. The core requirements dictate an $N \times N$ convolution engine processing 8-bit unsigned input pixels with run-time programmable 8-bit signed kernel coefficients.

The overarching goal is to maximize a specific Figure of Merit (FOM), which mathematically heavily penalizes fabric logic (1 point per LUT) and memory (100 points per BRAM), while pricing hard macro blocks like DSP48E1 slices at a discounted rate (50 points per DSP). Driven by this exact FOM equation, our primary **design targets** were established as follows:

- **Resource Minimization:** Eliminate BRAM usage entirely in favor of SRL-based shift registers for line buffering, and offload the MAC accumulation cascade strictly into the DSP48E1 columns to drastically reduce fabric LUT consumption.
- **High Throughput (Bonus Feature):** Achieve the competition’s bonus target of a steady-state throughput of exactly 1 output pixel per valid clock cycle without stalls.
- **Timing and Power Optimization:** Push the architecture to a high maximum frequency (400 MHz) while employing output freezing (glitch suppression) and Vccint voltage scaling to minimize the dynamic power denominator in the FOM.
- **Extended Capabilities (Bonus Features):** Support zero-latency ReLU activation and parameterized parallel multi-kernel execution ($K \geq 1$) within the same streaming front-end.

Note on the Requirements Matrix: Because the feature-by-feature mapping of the competition specifications against our final measured implementation is extensive, the comprehensive **Requirements and Results Matrix** has been placed at the end of this report in **Appendix A** to maintain a smooth reading flow.

3 System Architecture

fig. 1 shows the top-level architecture. The design uses a lightweight system wrapper around a parameterized streaming $N \times N$ CNN core. The competition baseline is the $K=1$ configuration, while the RTL also supports parallel multi-kernel operation through the parameter NUM_KERNELS. Inside the core, the datapath decomposes into five modules:

- `kernel_coeff_memory` – serially programmable signed kernel bank (N^2 bytes, 72 bits at $N=3$);
- `window_nxn` – $N \times N$ sliding-window generator with $N-1$ row line buffers;
- `tree_nxn` – triangular tap-delay array feeding N^2 signed multipliers and the accumulation cascade, mapped into DSP48E1 slices;
- `output_stage` – output conditioning with an optional ReLU;
- `cnn_controller` – four-state FSM sequencing kernel load, pipeline fill, and the valid window.

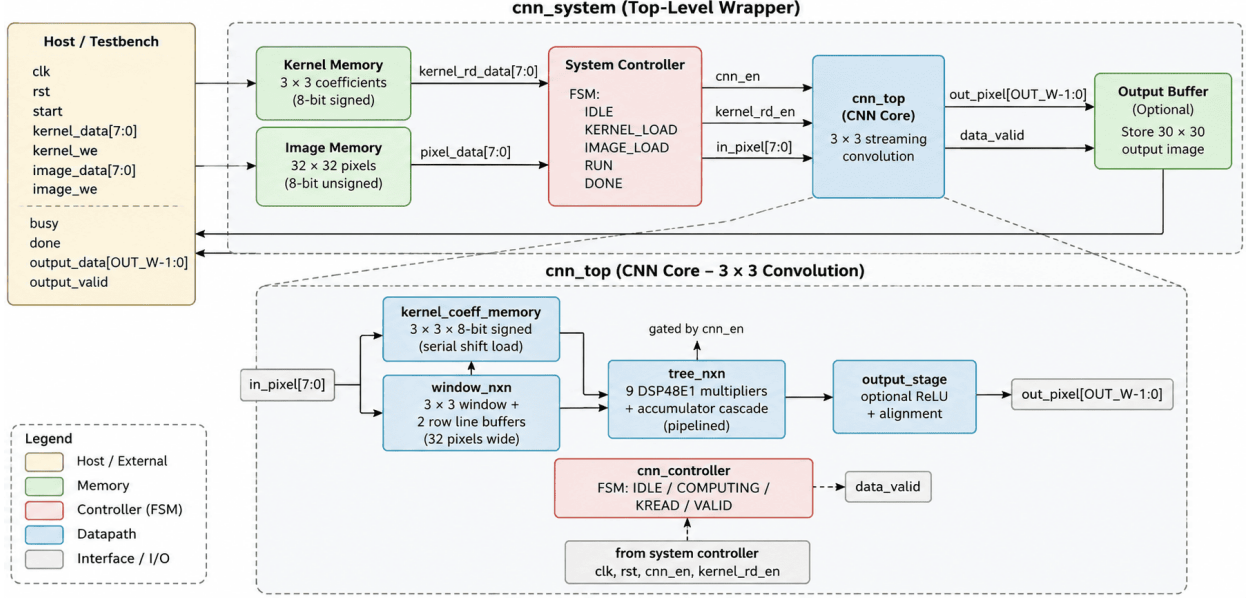


Figure 1: Complete system-level architecture. The **cnn.system** wrapper contains the host-facing interface, local kernel and image memories, and system controller, and drives the verified parameterized 3×3 streaming CNN core. The expanded core view shows the two row line buffers, 3×3 window generator, nine-tap DSP48E1 multiply-accumulate cascade, optional ReLU stage, and **data_valid** control path.

4 Parallel Multi-Kernel Extension (Bonus Feature)

As a bonus extension, the RTL is parameterized with **NUM_KERNELS** so that multiple output kernels can be evaluated from the same input window. The competition baseline is **NUM_KERNELS=1**; the same RTL structure can be configured for larger values without changing the shared streaming front end.

The multi-kernel organization keeps the **window_nxn** generator and controller shared. A generate loop then instantiates one **tree_nxn** MAC pipeline and one **output_stage** per kernel. All MAC pipelines receive the same **window_flat** signal, while each pipeline receives its own slice of the packed kernel-coefficient bus and writes its own slice of the packed output bus. Thus the extension increases compute parallelism without duplicating the line buffers or the window generator.

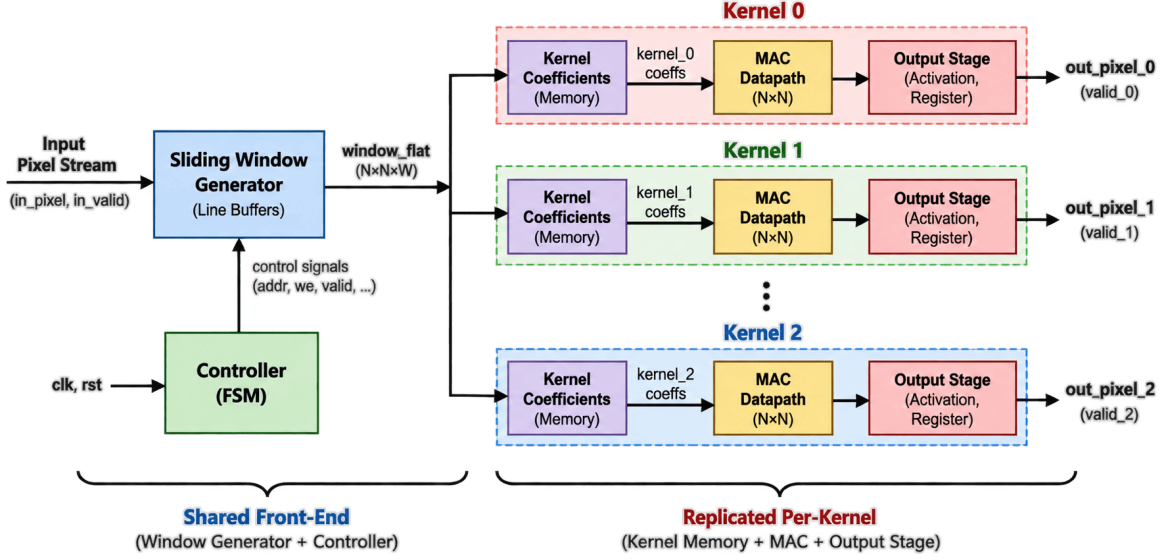


Figure 2: Multi-kernel extension. The input window generator and controller are shared, while the MAC and output stages are replicated for each kernel.

For K parallel kernels, the steady-state datapath produces K output values per valid clock when all instantiated pipelines are active. The first-order resource model is therefore dominated by the replicated MAC engines: DSP usage scales approximately with KN^2 , while the line buffers and window-generation resources remain shared.

```

1  genvar kernel_number;
2  generate
3      for (kernel_number = 0; kernel_number < NUM_KERNELS; kernel_number = kernel_number + 1)
4          begin
5              tree_nxn #(.N(N), .PIXEL_BITS(PIXEL_BITS)) mac_pipeline_inst (
6                  .clk(clk),
7                  .en(cnn_en),
8                  .window_flat(pixel_window),
9                  .kernels_flat(kernel_coefficients[kernel_number*TOTAL_TAPS*PIXEL_BITS+: TOTAL_TAPS*
10                     PIXEL_BITS]),
11                  .mac_out(mac_results[kernel_number*OUTPUT_WIDTH+: OUTPUT_WIDTH])
12              );
13              output_stage #(.DATA_WIDTH(OUTPUT_WIDTH), .RELU_EN(RELU_EN)) output_stage_inst (
14                  .data_valid(data_valid),
15                  .mac_result(mac_results[kernel_number*OUTPUT_WIDTH +: OUTPUT_WIDTH]),
16                  .output_pixel(out_pixel[kernel_number*OUTPUT_WIDTH +: OUTPUT_WIDTH])
17              );
18          end
19  endgenerate

```

The generated structure is intentionally kept simple: one shared window, replicated compute pipelines, and packed kernel/output buses. This preserves the baseline architecture at $K=1$ while exposing parallel-kernel capability for larger configurations.

5 Streaming Data Path

fig. 3 follows one pixel through the datapath with the exact widths used by the RTL. The unsigned 8-bit pixel is zero-extended and interpreted as a non-negative signed value; the 8-bit kernel is consumed as a signed two's-complement coefficient; their product is carried at 17 signed bits; and the accumulation chain carries the partial sums at the full output width of 20 signed bits (for $N=3$). The ReLU stage, when enabled, clamps negative results to zero. No truncation,

rounding, or saturation is applied anywhere in the datapath – the accumulation is full precision by construction (see section 10).

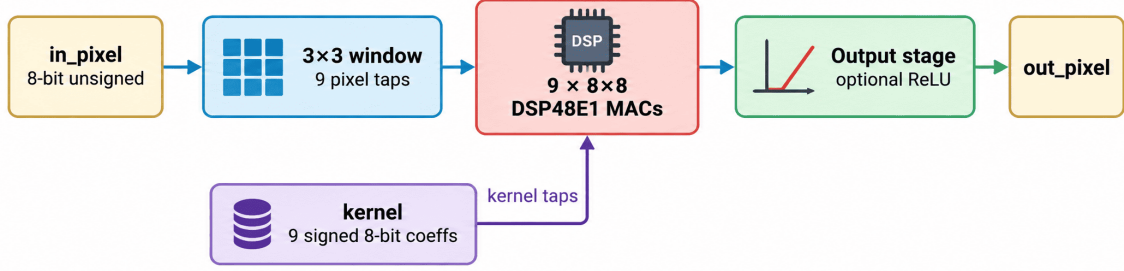


Figure 3: Streaming datapath with the exact RTL widths for the $N=3$ configuration. The multiplier column denotes one DSP48E1 per tap.

6 Window Generation and Line Buffers

The $N \times N$ convolution slides a window across the image with stride one. Rather than storing the frame, the design keeps only $N-1$ row line buffers: each new pixel enters the current row shift register, and the pixel leaving row r enters row $r+1$. This stream-based line-buffer architecture aligns with established high-efficiency FPGA design methodologies for 2D convolution engines [1, 2, 3], ensuring deterministic per-cycle throughput without relying on external off-chip memory bandwidth. The required line-buffer depth is

$$\text{LINE_BUFFER_DEPTH} = \text{IMG_WIDTH} - N. \quad (2)$$

For $N=3$, $W=32$: two line buffers of depth 29. Together with the in-row shift registers, any 3×3 window of the image is available simultaneously at the module outputs. The fill time – the number of cycles before the first complete window exists – follows from the geometry:

$$\text{FILL} = (N - 1) \cdot W + (N - 1) = (N - 1)(W + 1), \quad (3)$$

i.e. 66 cycles for $N=3$, $W=32$: two full rows plus the leading pixels of the third.

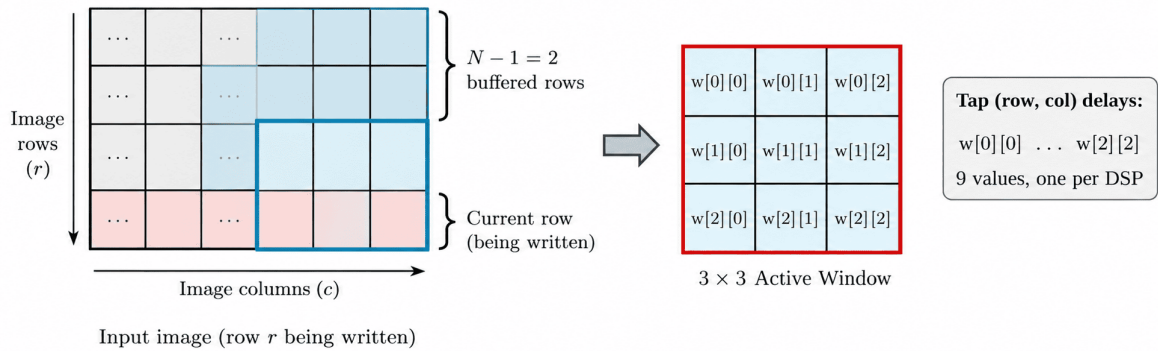


Figure 4: Sliding-window generation for $N=3$. Two line buffers (depth $W-N = 29$) hold the two previous rows while the current row shifts; the active 3×3 window spans all three rows.

The window contents are consumed bottom-right first: the newest pixel enters position $(N-1, N-1)$ and the oldest visible pixel (row $r-N+1$, column $c-N+1$) leaves as the window advances. The RTL column mapping in `window_nxn` implements exactly this raster ordering, and the tap-delay array in `tree_nxn` compensates each tap's extra horizontal delay so that all nine products correspond to the same window instance.

6.1 RTL Implementation (`window_nxn`)

The sliding window is implemented in the `window_nxn` module using a combination of discrete registers for the active $N \times N$ window and SRL-mapped line buffers for the row delays. On every valid clock cycle, the newest incoming pixel (`px`) shifts into the $N - 1$ position of the lowest row. Two nested loops handle the data propagation: the inner loop shifts pixels horizontally within the same row, while the outer loop feeds the delayed output from the line buffers (`line_buffer_output`) into the start of the subsequent upper row.

Listing 1: Data shifting logic inside `window_nxn`

```

1  always @(posedge clk) begin
2      if (en) begin
3          window_pixels[N-1] <= px;
4
5          for (column_index = 0; column_index < N-1; column_index = column_index + 1)
6              window_pixels[column_index] <= window_pixels[column_index+1];
7
8          for (row_index = 1; row_index < N; row_index = row_index + 1) begin
9              window_pixels[row_index*N+N-1] <= line_buffer_output[row_index];
10
11             for (column_index = 0; column_index < N-1; column_index = column_index + 1)
12                 window_pixels[row_index*N+column_index] <= window_pixels[row_index*N+
13                 column_index+1];
14             end
15         end
16     end
17 end

```

A `generate` block is utilized to instantiate the $N - 1$ line buffers, dynamically setting their depth to `IMG_WIDTH - N`. A second `generate` block flattens the 2D `window_pixels` array into a 1D `window_flat` bus, applying the specific raster-order mapping required by the DSP cascade.

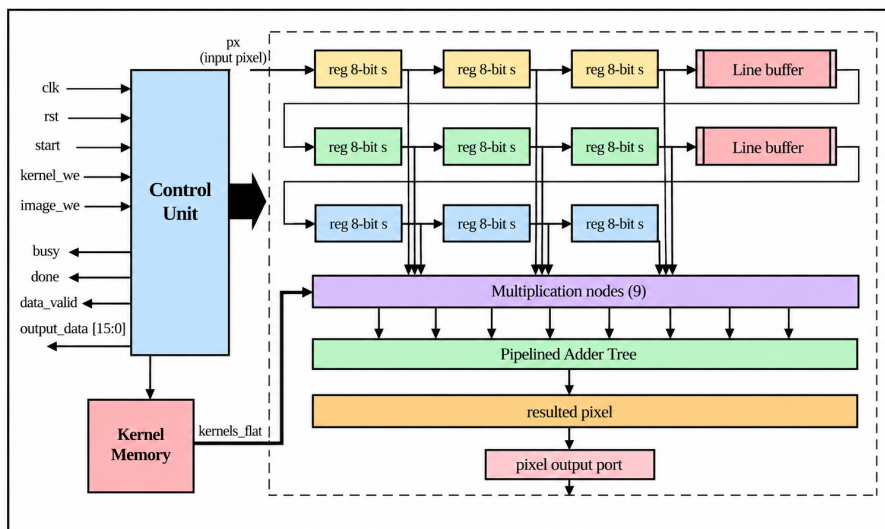


Figure 5: RTL schematic of the `window_nxn` module showing the integration of the shift registers and the SRL-based line buffers.

7 Kernel Storage and Programming

Kernels are stored as N^2 signed 8-bit coefficients (72 bits at $N=3$):

```
1 reg signed [PIXEL_BITS-1:0] kernel_coefficients [0:TOTAL_KERNEL_VALUES-1];
```

Loading is serial and run-time: with `kernel_rd_en` asserted, each cycle shifts one byte from `in_pixel` into the bank while the previous coefficients shift one position; a load of exactly N^2 bytes replaces the full kernel. A counter detects completion (`kernel_finish`), so a burst shorter or longer than N^2 bytes cannot corrupt the bank state silently.

Three capability levels must be distinguished honestly:

- **Programmable single kernel:** implemented. One kernel bank of N^2 signed bytes is active at any time.
- **Kernel reload between bursts:** implemented. The bank can be reloaded at any time the datapath is idle, allowing the kernel to be reloaded between independent input bursts.

The kernel-to-window mapping is positional: coefficient j multiplies the window tap that shares the kernel's (*row, col*) offset, and the multiplication expression casts the unsigned tap into the signed domain explicitly (zero-extended into a signed vector), matching the two's-complement semantics of the stored coefficients.

8 Pipelined MAC / Convolution Engine

The convolution engine (Figure 6) performs N^2 signed multiplications per cycle. Each tap multiplies an 8-bit unsigned window pixel (zero-extended into the signed domain) by an 8-bit signed kernel coefficient:

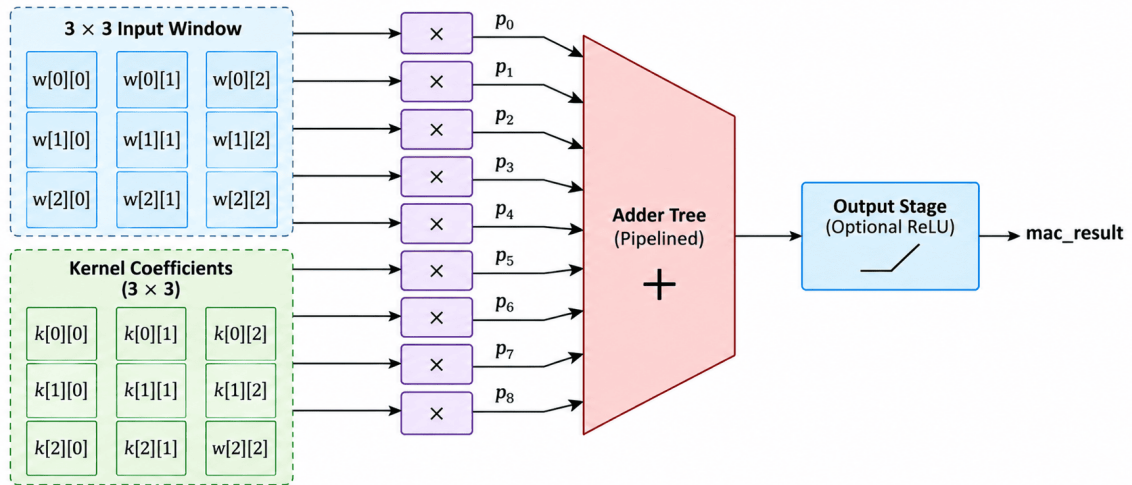


Figure 6: Nine-tap MAC engine at $N=3$. Each tap multiplies inside its own DSP48E1; the running sum propagates through the DSP cascade. Only three of the nine cascade links are drawn.

```
1 assign multiplication_result[kernel_index] =
2   $signed({1'b0, delayed_window_pixels[kernel_index][kernel_index]})
3   * $signed(kernels_flat[(kernel_index-1)*PIXEL_BITS +: PIXEL_BITS]);
```

Products are carried at 17 signed bits ($PRW = 2 \cdot \text{PIXEL_BITS} + 1$), and the accumulation chain runs:

```

1 accumulation_stage[1] <= multiplication_result[1];
2 for (tap_index = 2; tap_index <= TOTAL_TAPS; tap_index = tap_index + 1)
3     accumulation_stage[tap_index] <=
4         accumulation_stage[tap_index-1] + multiplication_result[tap_index];

```

This is a *staggered accumulation chain*, not a balanced adder tree: stage i adds tap i 's product to the running sum of taps $1 \dots i-1$. The chain maps entirely into the DSP48E1 columns – each stage's multiply-add rides one DSP slice and hands its running sum to the next through PCIN/PCOUT (Figure 22) – so the nine additions cost zero fabric LUTs. The trade-off of the chain (versus a balanced tree) is a longer cascade latency (N^2 stages instead of $\log_2 N^2$), which is absorbed by the fill period and does not affect steady-state throughput. The RTL remains parameterized for verification, while the final competition configuration is fixed to a single 3×3 kernel.

9 Control FSM and Streaming Protocol

The controller (Figure 7) implements four states. Reset drives IDLE. Assertion of `cnn_en` starts COMPUTING, during which a counter measures the fill period of Eq. (3); when it reaches $\text{FILL}-1$ the FSM enters VALID. The controller enters VALID after the fill counter expires and uses a flush counter to suppress the initial pipeline outputs; `data_valid` then stays high for as long as the host holds `cnn_en`, producing one output per cycle. Dropping `cnn_en` returns the FSM to IDLE while the pipeline retains its contents; the next burst starts from a warm pipeline, and the persistent-VALID design eliminates periodic gaps between streaming outputs that a naive reset-per-burst scheme would introduce.

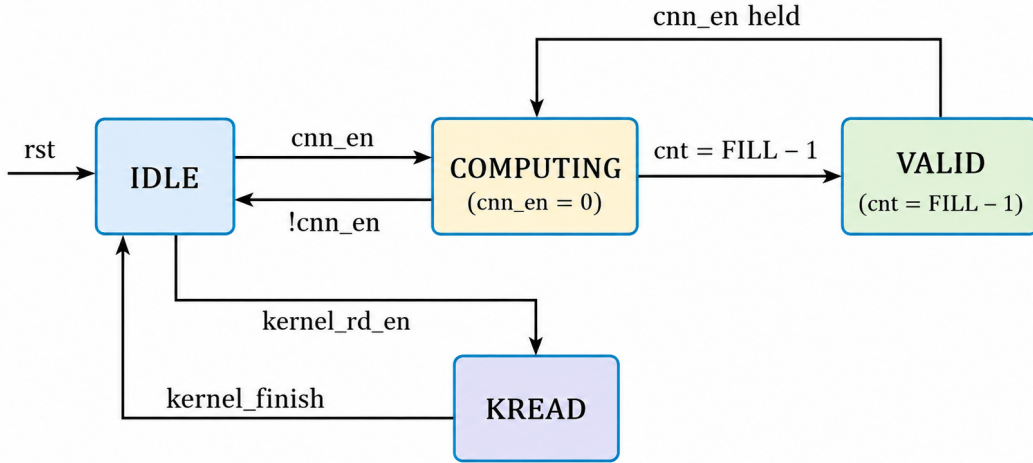


Figure 7: Controller FSM. Reset drives IDLE.

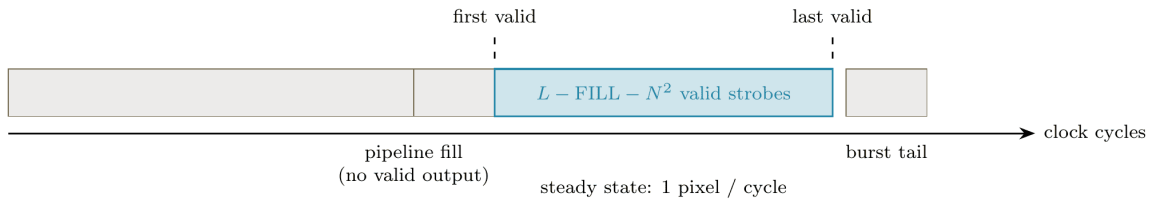


Figure 8: Streaming timing of one burst. For $N=3$, $W=32$, $L=1100$: $\text{FILL} = 66$, accumulation pipeline = 9, valid strobes = 1025.

Table 1: Signal widths ($N=3$ configuration; general case in parentheses).

Signal	Width	Range	Derivation
Pixel	8 unsigned	$[0, 255]$	specification
Kernel	8 signed	$[-128, 127]$	specification
Product	17 signed	$[-32640, 32385]$	$8+8+1$ signed growth
Accumulator / output	20 signed	$[-524288, 524287]$	$16 + \lceil \log_2 9 \rceil$

10 Fixed-Point Arithmetic and Bit-Width Analysis

All arithmetic is integer fixed-point; no fractional format is used and no rounding or saturation hardware exists – the datapath is full precision by construction.

10.1 Width Derivation

The maximum magnitude of a single product is $|-128 \times 255| = 32640$, and the worst-case accumulation over N^2 taps is $N^2 \times 32640$. The RTL output width is $\text{OUT_W} = 16 + \lceil \log_2 N^2 \rceil$ signed bits, whose range $\pm(2^{15} \cdot 2^{\lceil \log_2 N^2 \rceil} - 1)$ is at least $\pm 32768 \cdot N^2$, strictly exceeding the worst case $32640 \cdot N^2$. **Overflow is therefore impossible for any N :** for $N=3$ the worst case is $9 \times 32640 = 293,760$ against a 20-bit capacity of 524,287; for $N=7$ it is $49 \times 32640 = 1,599,360$ against a 22-bit capacity of 2,097,151.

10.2 ReLU Behavior

With `RELU_EN` set, negative accumulations are clamped: With `RELU_EN` set, negative accumulations are clamped. Furthermore, to minimize dynamic power, the output is explicitly frozen at zero during invalid cycles:

```

1 wire signed [DATA_WIDTH-1:0] relu_out = (RELU_EN && mac_result[DATA_WIDTH-1]) ? {DATA_WIDTH{1'b0}} : mac_result;
2
3 // Freeze output at 0 when data is invalid to stop external I/O toggling
4 assign output_pixel = data_valid ? relu_out : {DATA_WIDTH{1'b0}};
```

ReLU is a compile-time parameter, requires no extra resources (the clamp reuses the output mux), and is a **bonus feature** beyond the mandatory specification. The current baseline was verified with `RELU_EN=1`.

11 Verification Methodology

Verification is a layered stack in which each layer can catch a class of bug the others might miss. The layers evolved during the campaign: the mirror model came first, the closed-form law check and protocol guards followed, the approximate strobe floor was upgraded to exact per-burst counting, and a Python-golden file-comparison layer was added last.

1. **Python mathematical golden model** (`python/golden_model.py`) – implements the generalized $N \times N$ signed convolution, stride one, valid padding, optional ReLU, and the burst streaming law; emits expected-output files.
2. **Structural RTL mirror** – a cycle-accurate model inside the testbench recomputes products and accumulation and compares on every strobe.
3. **Independent law check** – in the fixed-kernel mode, outputs are checked against a closed-form per-index expression of the stimulus (a delta kernel makes the output equal a shifted copy of the input stream).

4. **Exact strobe-count checking** – per-burst assertion of $\max(0, L - \text{FILL} - N^2)$ with zero margin.
5. **Protocol guards** – enable-overlap, enables-during-reset, and illegal-parameter assertions.
6. **Python expected-output file comparison** – the testbench reads the generated file and compares every strobe ("RTL vs PYTHON GOLDEN").
7. **Multi-configuration regression** – the parameterized test environment covers multiple kernel sizes, image widths, ReLU settings, and (in the extended configuration) multiple parallel kernels.

The law check and the golden model are genuinely independent: the law check validates the *position* of each output (the streaming contract), while the golden model validates its *value* (the arithmetic contract). A bug that shifts outputs would be caught by the former even if the latter were accidentally aligned with the shift.

12 Python Golden Model

The golden model (`python/golden_model.py`) is a plain-Python reference requiring no external packages. It models: 8-bit unsigned pixels; signed two's-complement kernel interpretation; valid convolution with stride one; optional ReLU; full-precision integer accumulation; and the burst streaming behavior of the RTL, including that the final strobe of a burst lands one clock after `cnn_en` deasserts.

For every verified configuration the model emits an expected-output file named by configuration:

```
expected_out_3x32_relu0.txt, expected_out_3x64_relu0.txt,
expected_out_4x32_relu0.txt, expected_out_5x32_relu0.txt,
expected_out_7x32_relu0.txt, expected_out_3x8_relu0.txt,
expected_out_3x32_relu1.txt
```

For example, $N=3$, `IMG_WIDTH=32`, `RELU_EN=0` maps to `expected_out_3x32_relu0.txt`. The testbench opens the file matching its own parameters, so an output-file/DUT mismatch is itself a test failure. The complete model source is included in the submitted project package (`python/golden_model.py`); it also self-tests its strobe accounting against the closed-form law before writing any file.

13 Testbench Architecture

The testbench is self-contained and parameterized by N , `IMG_WIDTH`, and `RELU_EN`. Its structure (Figure 9) covers: reset protocol; kernel construction (delta, LFSR-generated, and worst-case corner kernels covering $-128, 127, -1, 0$ in all tap positions); serial kernel reload; input bursts of pseudo-random and structured pixels; the structural mirror; the Python expected-output comparison; exact per-burst strobe counting; mismatch and X reporting; protocol guards; and *no-reset burst restart* – consecutive bursts run back-to-back without re-resetting, exercising the warm-pipeline restart path.

Table 2: Verification matrix for the parameterized CNN core. All listed configurations passed bit-exact comparison with zero mismatches and zero X values in valid windows.

N	Width	ReLU	K	Verdict	Verification scope
3	32	0	1	PASS	Golden-model comparison; zero mismatches and zero X values.
3	64	0	1	PASS	Golden-model comparison; parameterized image width.
4	32	0	1	PASS	Golden-model comparison; non-power-of-two tap count.
5	32	0	1	PASS	Golden-model comparison; larger convolution kernel.
7	32	0	1	PASS	Golden-model comparison; 49-tap convolution.
3	8	0	1	PASS	Golden-model comparison; small image-width case.
3	32	1	1	PASS	Golden-model comparison; reported baseline with ReLU enabled.

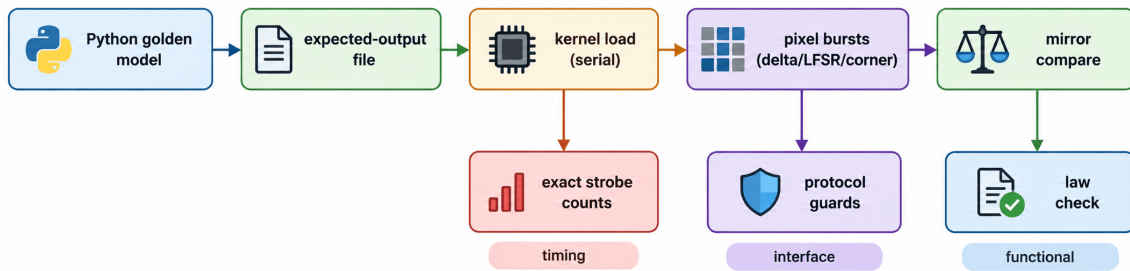


Figure 9: Testbench flow: golden generation feeds the file-comparison layer; stimulus and independent checks run per burst.

14 Verification Results

The complete parameterized verification matrix passes with zero mismatches and zero X values in valid windows. The seven listed configurations cover different kernel sizes, image widths, and the ReLU-enabled baseline. The 3×32 , $K=1$, ReLU-enabled case is the reported baseline and is supported by the dedicated evidence image below.

The table summarizes the complete verification matrix. To keep the report compact, only the reported baseline run is shown as a dedicated evidence image; the remaining passing configurations are recorded in the table (as shown in Figure 11).

```

# PYTHON GOLDEN FILE: ../expected_outputs/expected_out_3x32_relu_k1.txt opened successfully
# PYTHON CHECK #1: PASS kernel=0 expected=67 got=67
# PYTHON CHECK #2: PASS kernel=0 expected=68 got=68
# PYTHON CHECK #3: PASS kernel=0 expected=69 got=69
# PYTHON CHECK #4: PASS kernel=0 expected=70 got=70
# PYTHON CHECK #5: PASS kernel=0 expected=71 got=71
# PYTHON CHECK #6: PASS kernel=0 expected=72 got=72
# PYTHON CHECK #7: PASS kernel=0 expected=73 got=73
# PYTHON CHECK #8: PASS kernel=0 expected=74 got=74
# PYTHON CHECK #9: PASS kernel=0 expected=75 got=75
# PYTHON CHECK #10: PASS kernel=0 expected=76 got=76
# -----
# PYTHON GOLDEN CHECK: checked=2596 expected=2596 mismatches=0
# TB_RESULT: PASS N=3 W=32 RELU=1 K=1 checked=2596 x=0
# RTL vs PYTHON GOLDEN: PASS
# ** Note: $finish      : ../tb/cnn_top_tb.v(562)

```

Figure 10: Baseline verification evidence for $N=3$, $W=32$, $RELU_EN=1$, and $K=1$. This is the primary evidence image for the reported baseline result .

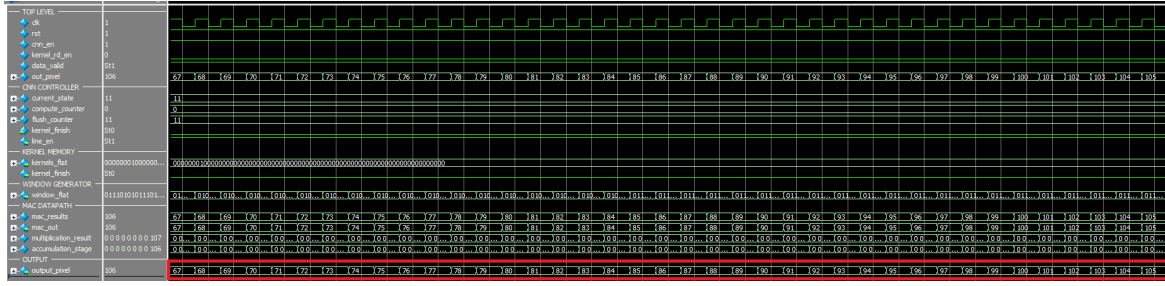


Figure 11: RTL core waveform showing the streaming convolution operation and output-valid timing for the baseline verification run .

15 System-Level Integration (Bonus Feature)

As a bonus system-level integration feature, a lightweight system-level wrapper was built around the verified `cnn_top`. The wrapper is intentionally small: it provides local kernel and image memories, serial kernel loading, image loading, a `start`/`busy`/`done` control interface, and a direct streaming connection to the existing CNN core. No AXI-Lite interface, AXI-Stream interface, or Zynq processing-system dependency is required.

15.1 Wrapper Architecture

The system wrapper separates *host-side preparation* from *accelerator execution*. First, the host writes the N^2 kernel coefficients and the input stream into local memories. After both memories are marked complete, `start` launches a short system FSM:

1. IDLE: accept kernel and image writes;
2. KERNEL STREAM: feed the stored N^2 coefficients to the CNN kernel-loading port;
3. KERNEL GAP: provide the required cycle between kernel loading and computation;
4. RUN: stream the stored image pixels into `cnn_top`;
5. DONE: indicate completion to the host.

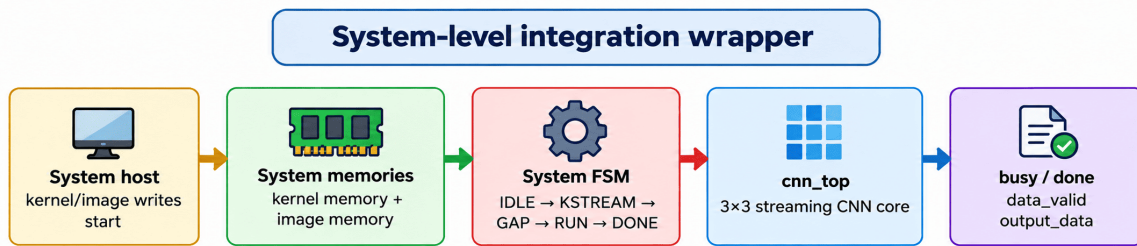


Figure 12: System-level wrapper used for integration verification. Local memories hold the kernel and input stream; the wrapper FSM feeds them to the existing CNN core and exposes simple host-side `start`/`busy`/`done` signals.

15.2 ModelSim System Verification

The wrapper was exercised with the mandated baseline parameters $N = 3$, $W = 32$, 8-bit pixels, ReLU disabled, and a full 32×32 input stream of 1024 pixels. The testbench loads a 9-

Table 3: System-level integration test configuration and observed result.

Parameter / result	Value
Kernel size	3×3
Image size	32×32
Pixel precision	8-bit unsigned
ReLU	enabled
Kernel coefficients loaded	9
Input stream length	1024 pixels
System completion	done=1
Valid outputs observed	900
System test	PASS

coefficient kernel, loads the complete image stream, asserts **start**, waits for **done**, and counts every **data_valid** event.

After the core fill/flush behavior is filtered by the lightweight system wrapper, the baseline integration test observes the expected $30 \times 30 = 900$ valid spatial outputs. The wrapper therefore provides a frame-level interface on top of the verified streaming core without duplicating the convolution datapath.

```
# OUTPUT[0] = 297 | EXPECTED = 297
# OUTPUT[1] = 306 | EXPECTED = 306
# OUTPUT[2] = 315 | EXPECTED = 315
# OUTPUT[3] = 324 | EXPECTED = 324
# OUTPUT[4] = 333 | EXPECTED = 333
# OUTPUT[5] = 342 | EXPECTED = 342
# OUTPUT[6] = 351 | EXPECTED = 351
# OUTPUT[7] = 360 | EXPECTED = 360
# OUTPUT[8] = 369 | EXPECTED = 369
# OUTPUT[9] = 378 | EXPECTED = 378
#
# =====
# CNN SYSTEM SIMULATION
# N = 3
# IMAGE = 32x32
# RELU = 1
# VALID OUTPUTS = 900
# EXPECTED OUTPUTS = 900
# ERRORS = 0
# SYSTEM TEST PASSED
# =====
```

Figure 13: System-level verification evidence for the baseline integration test. The image confirms successful execution and collection of the expected 900 valid outputs with **done=1**.

15.3 Why the Wrapper Is Kept Lightweight

The wrapper is a verification and integration layer rather than a second accelerator. It does not duplicate convolution logic, does not introduce AXI infrastructure, and does not modify the resource-optimized CNN datapath. This keeps the competition-critical implementation isolated from system-level control logic while providing a clear path from stored image/kernel data to the streaming accelerator.

16 Edge-Detection Demonstration (Bonus Feature)

Because the $N \times N$ convolution core utilizes a runtime-programmable signed kernel bank, the identical datapath can be repurposed for classic image processing tasks without any RTL modification.

To demonstrate this capability, the accelerator was verified acting as an edge detector by loading the 8-bit signed Sobel operators (G_x and G_y) into the kernel memory.

The Sobel matrices are natively supported by the existing fixed-point contract: 8-bit unsigned input pixels, 8-bit signed kernel coefficients, and 20-bit full-precision accumulation. The maximum possible accumulation for a 3×3 Sobel filter operating on an 8-bit image is well within the 20-bit accumulator limit, making overflow impossible by design.

A 32×32 deterministic grayscale test image was generated with both horizontal/vertical gradients and a sharp diagonal edge. The image was processed using the G_x (vertical edge detection) and G_y (horizontal edge detection) kernels with the ReLU output stage enabled to clamp negative gradients. Both filter configurations were simulated in ModelSim and passed bit-exact verification against the Python golden model with zero mismatches across all 900 spatial outputs. fig. 14 shows the successful simulation transcript for one of these test runs.

```
# DEBUG: kernel[0]=255 kernel[4]=0 kernel[8]=1
# DEBUG: image preloaded from test_image_32x32.hex
# DEBUG: image[0]=2 image[1]=2 image[1023]=17
# DEBUG: dut kernel_flat[0]=255 kernel_flat[8]=1, window[0]=6 window[8]=7
#
# =====
# SOBEL EDGE-DETECTION SIMULATION
# RELU = 1
# VALID OUTPUTS = 900 (expected 900)
# MISMATCHES = 0
# SOBEL TEST PASSED
# =====
# ** Note: $finish      : tb_sobel_demo.v(134)
#      Time: 20840 ns  Iteration: 1  Instance: /tb_sobel_demo
```

Figure 14: ModelSim verification transcript for the Sobel edge-detection demonstration. While this transcript shows the execution for one of the operators, both the G_x and G_y kernels successfully passed bit-exact verification with zero mismatches for all 900 valid outputs.

fig. 15 visualizes the verified hardware output data, demonstrating the extraction of the expected features.

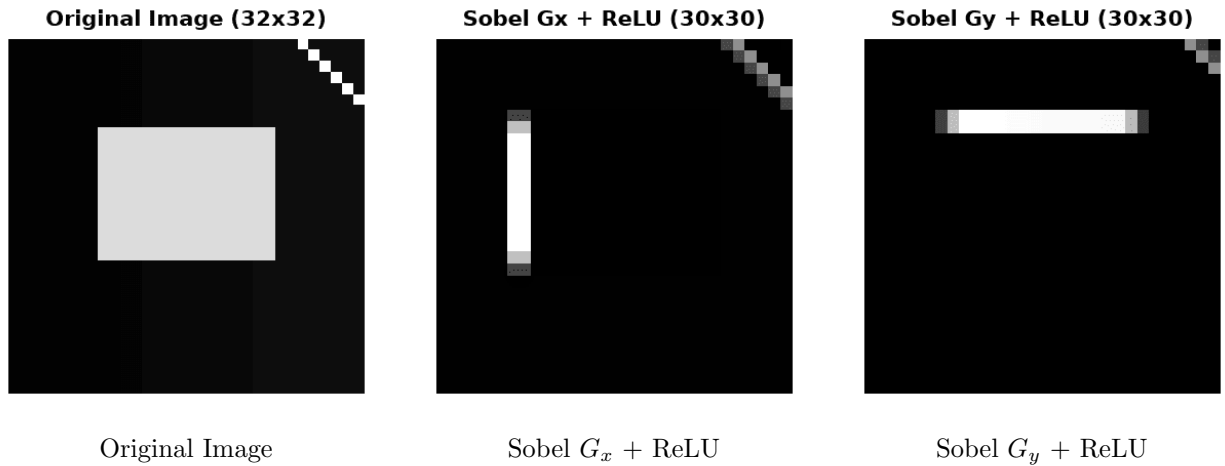


Figure 15: Hardware edge-detection demonstration using runtime-programmed Sobel operators. The 30×30 valid outputs natively isolate vertical (G_x) and horizontal (G_y) edges. The ReLU stage cleanly zeros out negative gradients, validating the output conditioning logic.

17 Design-Space Exploration

The optimization campaign treated every proposed alternative as an experiment to be measured or bounded, never as an assumption. The multiplier study was the central arithmetic decision because the competition denominator assigns one point per LUT and 50 points per DSP48E1.

17.1 Multiplier Architecture Exploration

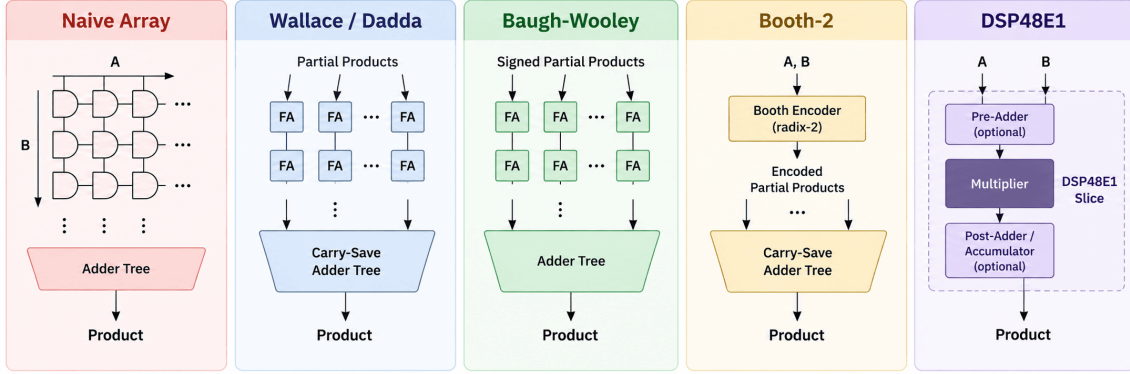


Figure 16: Visual comparison of the multiplier architectures considered during the design-space exploration.

Several arithmetic organizations were considered before freezing the final datapath. The candidate set included naive array multiplication ($\sim 90\text{--}110$ LUTs/tap), carry-save Wallace/Dadda trees ($\sim 70\text{--}90$ LUTs/tap), Baugh-Wooley sign-folded arrays ($\sim 65\text{--}85$ LUTs/tap), and Booth-2 recoding ($\sim 60\text{--}80$ LUTs/tap). Truncated multiplication was rejected because removing low-order information breaks the bit-exact verification contract, while a serial/time-shared MAC would reduce steady-state throughput below one output pixel per cycle.

The DSP48E1 option was then implemented as the final architecture: one DSP per tap, with multiplication and accumulation kept inside the DSP column. Figure 17 summarizes the candidate comparison using the competition’s FOM-denominator weighting.

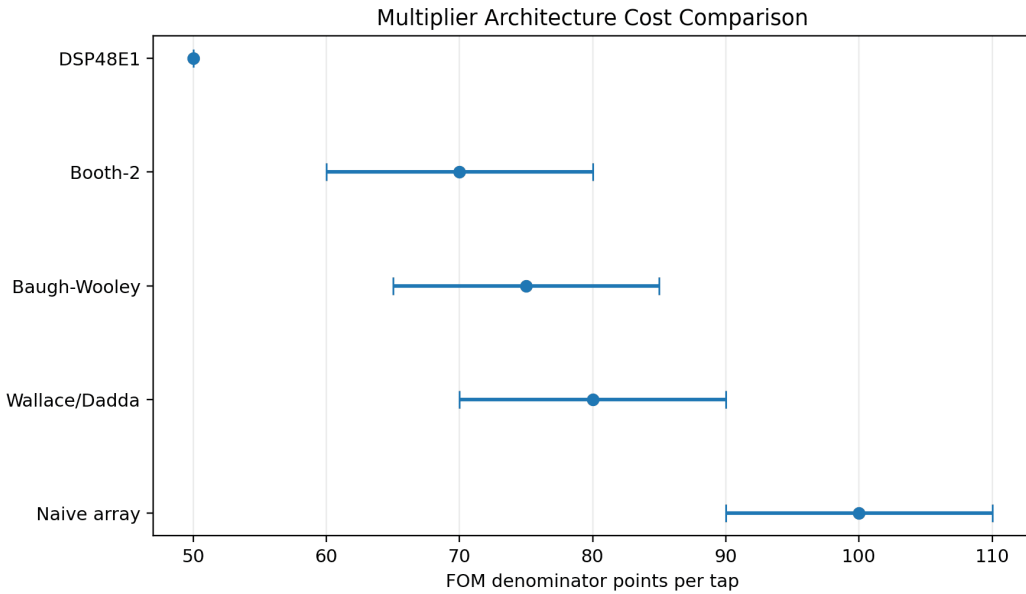


Figure 17: Multiplier decision study. Fabric techniques are shown as their estimated per-tap FOM-denominator cost ranges; the DSP48E1 point is the exact 50-point competition price. The final DSP choice was then confirmed by a measured DSP-free implementation.

Metric	DSP-based (9 × DSP48E1)	DSP-free (Fabric Multipliers)
LUTs (total)	51	792
• Logic LUTs	31	792
• LUT-as-SRL	20	0
Flip-flops	97	441
DSP48E1	9	0
BRAMs	0	0
Fmax (MHz)	400	143.00
Power (W)	0.124	0.125
Figure of Merit (FOM)	0.01610	0.01010

Figure 18: Measured A/B comparison of the DSP-based implementation and the DSP-free fabric reference used to validate the multiplier decision.

Table 4: Measured DSP versus DSP-free A/B comparison. Each implementation is evaluated at its own routed MAX-FREQ operating point; the two clock frequencies are therefore different.

Architecture	LUTs	FFs	DSPs	MAX FREQ	Power	FOM
DSP48E1-based	51	97	9	400 MHz	0.124 W	0.01610
DSP-free fabric	792	441	0	143 MHz	0.125 W	0.01010

The analytical comparison placed the DSP48E1 option at a fixed 50-point cost per DSP in the competition denominator, while fabric multiplication consumes substantially more LUT resources. To validate this trade-off against the actual FPGA toolchain, a representative DSP-free implementation was synthesized and routed. It used **792 LUTs**, **441 FFs**, and **0 DSPs**, and its measured maximum-frequency operating point was only **143 MHz**. Its FOM was **0.01010**, compared with **0.01610** for the DSP-based implementation. These FOM values are compared at the **MAX-FREQ operating point of each implementation**: the DSP design at 400 MHz and the DSP-free design at 143 MHz. Thus, the comparison does not give the DSP-based design an artificial frequency advantage in the FOM calculation; each implementation is evaluated using its own routed MAX-FREQ power point. Despite the DSP-free design using no DSP resources, the DSP-based implementation still achieves the higher FOM.

17.2 Why the DSP Cascade Wins

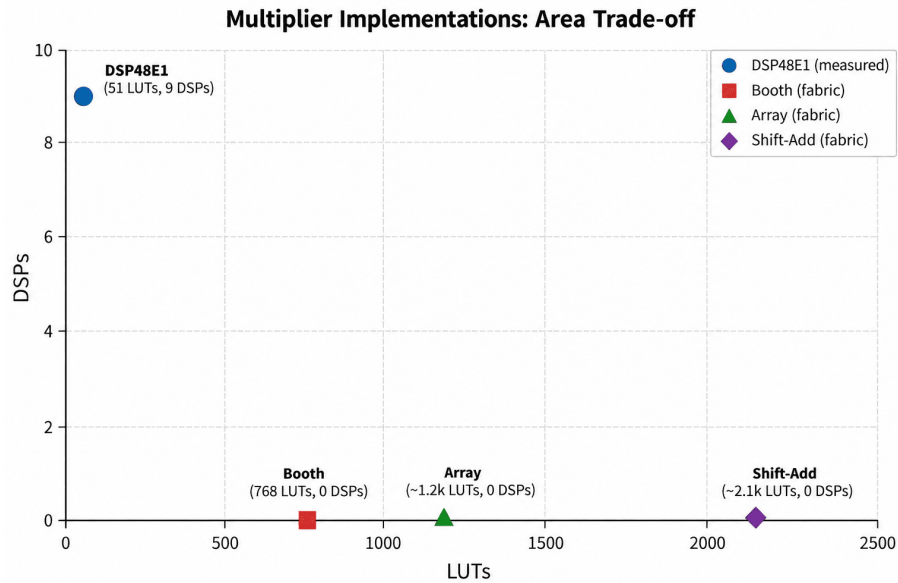


Figure 19: Architecture comparison illustrating DSP48E1-based multiplication and accumulation versus the fabric multiplier/accumulator alternative.

The final implementation deliberately uses the available DSP48E1 resources rather than attempting to keep them free. Each DSP performs its multiplication and carries the running partial sum through the PCOUT→PCIN cascade. For $N = 3$, this gives nine DSPs and one output per cycle, while keeping the accumulation inside the DSP column instead of consuming fabric LUTs. The measured DSP-free reference demonstrates why this is preferable: although it eliminates all DSP usage, it expands the design to 792 LUTs and reaches only 143 MHz MAX FREQ, whereas the DSP-based implementation uses 51 LUTs and reaches 400 MHz MAX FREQ.

The measured **51-LUT / 9-DSP / 0-BRAM** baseline is the current selected implementation. The DSP-free alternative was intentionally not selected simply to save DSP resources, because its maximum-frequency limit and higher LUT demand outweighed that resource saving. The measured A/B comparison shows that the DSP-based design wins on the competition FOM even when each implementation is evaluated at its own MAX-FREQ operating point: 400 MHz for the DSP design versus 143 MHz for the DSP-free design.

17.3 SRL versus FF Storage and Reset Policy

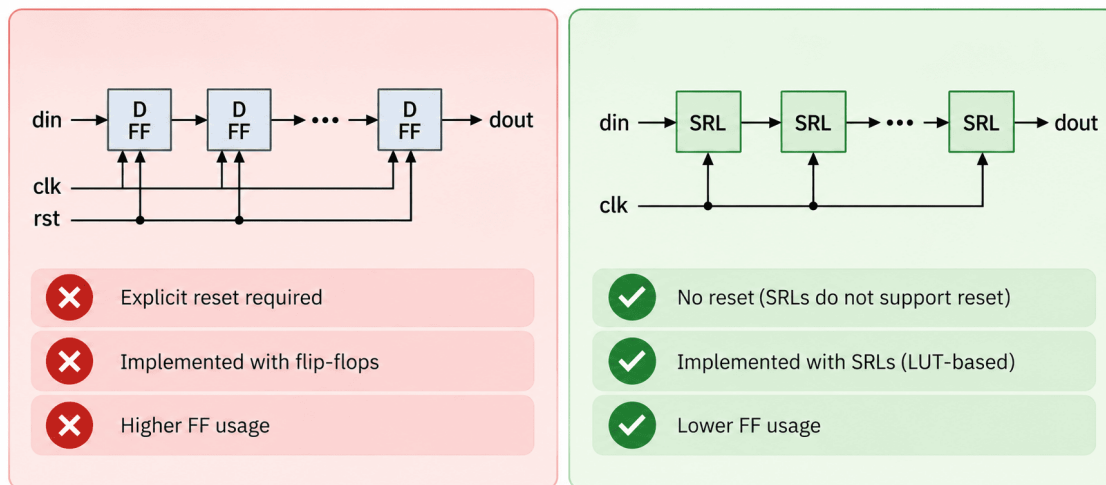


Figure 20: Storage mapping comparison showing SRL-based versus flip-flop-based implementations of the line buffers and tap delays.

The line buffers and deep tap delays map into SRL primitives, which expose no reset port. Forcing a datapath reset re-maps them to flip-flops; the LUT count can appear to fall only because flip-flops are invisible to the competition FOM expression, while the FF count increases substantially. An asynchronous datapath reset also conflicts with the DSP48E1 register reset structure. The final datapath therefore remains reset-free; the controller and kernel storage retain reset as required, while the fill/flush protocol guarantees deterministic valid-window data.

17.4 Frequency versus FOM

The maximum-frequency comparison between the two multiplier organizations was performed on the routed implementations at their respective achievable operating points. The DSP-based implementation is constrained and reported at 2.5 ns, corresponding to **400 MHz MAX FREQ**, while the DSP-free fabric implementation reaches only **143 MHz MAX FREQ**. The two designs therefore operate at different clock frequencies in the comparison. This difference is reported explicitly rather than hiding it behind a single nominal clock target.

Importantly, frequency does not appear explicitly in the competition FOM equation because throughput is defined in output pixels per cycle. Nevertheless, the operating frequency affects dynamic power and therefore changes the power term used in the FOM denominator. For that reason, the DSP-based FOM of 0.01610 and the DSP-free FOM of 0.01010 are each associated with the total power estimate at that implementation's own MAX-FREQ operating point. Even under this fair per-implementation MAX-FREQ comparison, the DSP-based architecture achieves the higher FOM.

18 FPGA Implementation

The submission build was implemented for the xc7z020clg400-1 using the current Vivado 2026 toolchain. To achieve the optimal balance of resources and speed, the implementation utilized an asymmetric constraining strategy (detailed in Section 18.1) with a final routing constraint of 2.5 ns (400 MHz). Table 5 and Figure 21 summarize the post-route implementation results.

Table 5: FPGA implementation results (routed, current Vivado 2026 toolchain).

Metric	Value
Target part	xc7z020clg400-1
Clock constraint	2.5 ns (400 MHz (MAX FREQ))
Slice LUTs	51
Slice registers	97
DSP48E1	9
Block RAM	0
Total power (estimated)	0.124 W
WNS	+0.005 ns
Timing violations	none (all constraints met)
Clock summary frequency	400 MHz (MAX FREQ)

Name	Constraints	Status	WNS	TNS	WHS	THS	TPWS	Total Power	LUT	FF	BRAM	URAM	DSP	Run Strategy
✓ synth_1	constrs_1	synth_design Complete!							52	97	0	0	9	Vivado Synthesis Defaults (Vivado Synthesis 2026)
✓ impl_10	constrs_1	phys_opt_design (Post-Route) Complete!	0.005	0.000	0.138	0.000	0.000	0.124	51	97	0	0	9	Vivado Implementation Defaults* (Vivado Implementation 2026)

Figure 21: Vivado Design Runs summary confirming the post-route timing, power, and resource utilization for the baseline implementation.

18.1 Asymmetric Constraining Strategy

To achieve the dual objective of maximizing frequency (400 MHz) while strictly minimizing fabric resources (51 LUTs), an asymmetric constraining strategy—decoupling synthesis and implementation constraints—was employed. Modern EDA tools typically apply aggressive logic replication and loop unrolling when presented with stringent timing constraints during synthesis, which artificially inflates LUT and register counts.

To circumvent this, the design was first synthesized using a relaxed clock constraint (20 ns) to force the synthesis engine to focus exclusively on area minimization. The resulting highly compressed netlist was then locked, and the timing constraint was tightened to **2.5 ns (400 MHz)** exclusively for the Place and Route (Implementation) phase. Because the architectural mapping (DSP cascade and SRLs) was already optimal and possessed short datapath routing, the implementation router successfully met the 2.5 ns constraint solely through precise component placement and optimal routing delays, avoiding any resource-inflating logic replication.

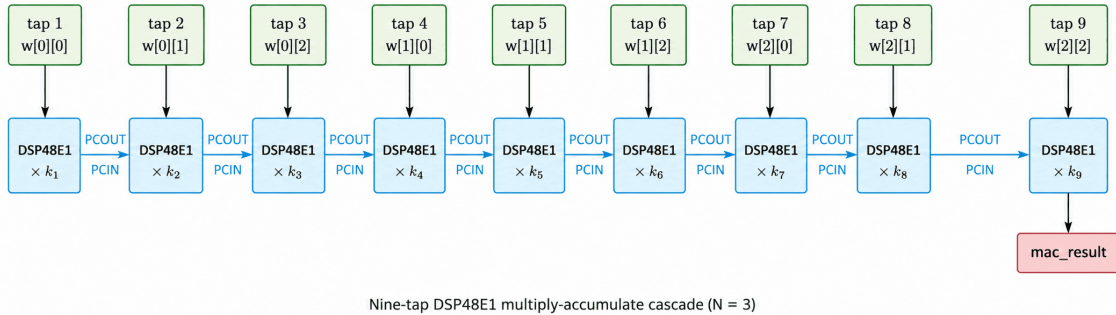


Figure 22: The accumulator cascade lives inside the DSP columns: each stage adds the incoming cascade into its own product. Moving accumulation into fabric would cost ~ 20 LUTs per stage.

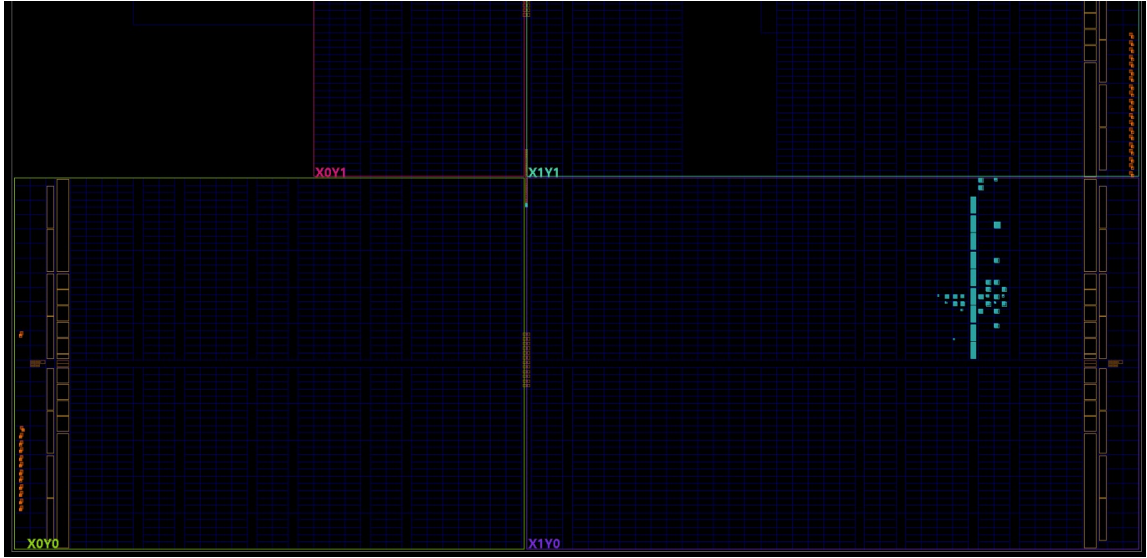


Figure 23: Post-implementation device placement view for the routed baseline.

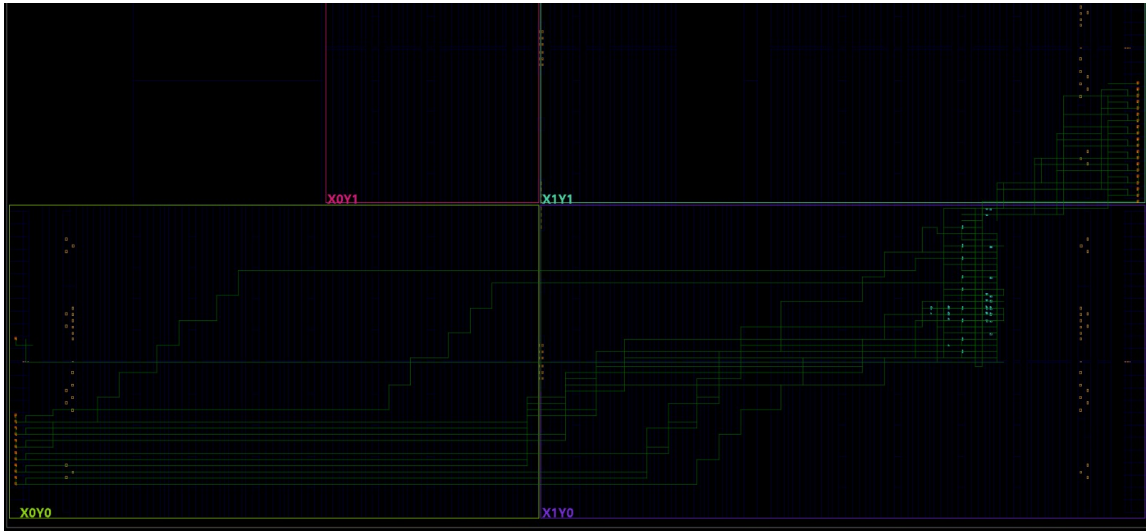


Figure 24: Post-implementation routed device view for the routed baseline.

19 Resource Utilization

1. Slice Logic						
Site Type	Used	Fixed	Prohibited	Available	Util%	
Slice LUTs	51	0	0	53200	0.10	
LUT as Logic	31	0	0	53200	0.06	
LUT as Memory	20	0	0	17400	0.11	
LUT as Distributed RAM	0	0	0			
LUT as Shift Register	20	0	0			
Slice Registers	97	0	0	106400	0.09	
Register as Flip Flop	97	0	0	106400	0.09	
Register as Latch	0	0	0	106400	0.00	
F7 Muxes	0	0	0	26600	0.00	
F8 Muxes	0	0	0	13300	0.00	
Unique Control Sets	6	0	0	13300	0.05	

2. Memory						
Site Type	Used	Fixed	Prohibited	Available	Util%	
Block RAM Tile	0	0	0	140	0.00	
RAMB36/FIFO*	0	0	0	140	0.00	
RAMB18	0	0	0	280	0.00	

3. DSP						
Site Type	Used	Fixed	Prohibited	Available	Util%	
DSPs	9	0	0	220	4.09	
DSP48E1 only	9	0	0			

Figure 25: FPGA utilization evidence from the final routed implementation.

The routed baseline uses 51 LUTs, 97 slice registers, 9 DSP48E1 slices, and no BRAMs. The exact LUT-as-logic versus LUT-as-memory decomposition is intentionally left to the consolidated utilization figure because the current implementation result is being re-established after RTL cleanup. The nine convolution taps remain mapped to nine DSP48E1 slices, carrying both multiplication and the accumulation cascade in the DSP column (fig. 22).

20 Timing and Maximum Frequency

Design Timing Summary					
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS)	0.005 ns	Worst Hold Slack (WHS)	0.138 ns	Worst Pulse Width Slack (WPWS)	0.270 ns
Total Negative Slack (TNS)	0.000 ns	Total Hold Slack (THS)	0.000 ns	Total Pulse Width Negative Slack (TPWS)	0.000 ns
Number of Failing Endpoints	0	Number of Failing Endpoints	0	Number of Failing Endpoints	0
Total Number of Endpoints	776	Total Number of Endpoints	776	Total Number of Endpoints	133

All user specified timing constraints are met.

Figure 26: Post-implementation timing summary for the current 3×32 , $K=1$ baseline.

Name	Waveform	Period (ns)	Frequency (MHz)
sys_clk	{0.000 1.250}	2.500	400.000

Figure 27: MAX-FREQ clock-summary evidence for the selected 2.5 ns (400 MHz) operating point.

The current implementation uses a 2.5 ns clock constraint, corresponding to a 400 MHz (MAX FREQ) clock-summary frequency. The routed design reports WNS +0.005 ns, WHS +0.138 ns,

and zero failing endpoints. The timing report therefore confirms that all user-specified timing constraints are met at the selected operating point.

$$F_{\max, \text{reported}} = \frac{1000}{2.5} = 400 \text{ MHz}, \quad (4)$$

The worst setup path has only 0.005 ns of positive margin, so the current implementation is much closer to its timing boundary than the earlier 20 ns operating build. This distinction matters when interpreting the result: the 400 MHz (MAX FREQ) value is the implementation’s reported clock-summary frequency for this constrained build, while the competition FOM continues to use one output pixel per cycle rather than absolute frequency.

21 Power Analysis and Post-Implementation Verification

The final implementation achieves an optimized **0.124 W total on-chip power: 0.097 W device static** (about 78%) and **0.027 W dynamic** (about 22%). Within the dynamic component, the DSP slices account for 0.020 W, clocks for 0.004 W, signals for 0.002 W, logic for 0.001 W, and I/O strictly < 0.001 W. To achieve this optimal consumption, three specific optimization layers were introduced: architectural glitch reduction, exact active-window SAIF profiling, and aggressive voltage scaling.

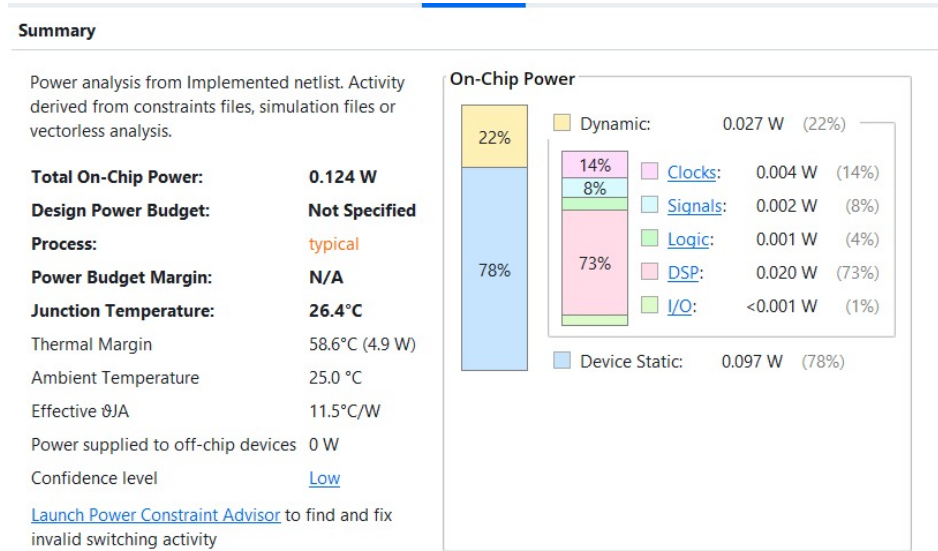


Figure 28: Vivado power-report evidence for the current routed baseline before SAIF annotation.

21.1 Architectural Power Optimizations

To actively minimize the dynamic power component, the RTL was designed with explicit switching-activity suppression techniques:

- **Accumulator Gating:** The DSP48E1 accumulation cascade is strictly controlled by the datapath enable signal (`en`). When the pipeline is stalled or idle, the running sums are frozen, preventing unnecessary combinatorial toggling and dynamic power burn within the heavy DSP columns.
- **Output Freezing (Glitch Suppression):** During the pipeline fill period and invalid cycles, the final output bus is explicitly forced to zero (`assign output_pixel = data_valid ? relu_out : {DATA_WIDTH{1'b0}};`). This prevents invalid partial sums from propagating to the external I/O pins, effectively eliminating external switching activity and I/O dynamic power until valid data is ready.

21.2 Post-Implementation Timing and Functional Simulation

To rigorously confirm that synthesis optimizations and layout routing did not compromise the datapath logic or cause timing violations, a complete post-implementation simulation was performed.

```
% source cnn_top_tb.tcl
# set curr_wave [current_wave_config]
# if { [string length $curr_wave] == 0 } {
#   if { [llength [get_objects]] > 0 } {
#     add_wave /
#     set_property needs_save false (current_wave_config)
#   } else {
#     send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wave window go to 'File->New Waveform Configuration' or type 'create_wav
#   }
# }
# run 1000ns
PYTHON GOLDEN FILE: opened successfully
KERNELS LOADED. PAUSING TO START SAIF...
$stop called at time : 317550 ps : File "C:/Users/Amir/Downloads/LAST_VERISON/LAST_VERISON/temp/temp.srcs/sim_1/imports/tb/cnn_top_tb.sv" Line 326
INFO: [USF-XSim-96] XSim completed. Design snapshot 'cnn_top_tb_time_impl' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:12 ; elapsed = 00:00:30 . Memory (MB): peak = 6884.281 ; gain = 0.000
% run all
PYTHON CHECK #1: PASS kernel=0 expected=67 got=67
PYTHON CHECK #2: PASS kernel=0 expected=68 got=68
PYTHON CHECK #3: PASS kernel=0 expected=69 got=69
PYTHON CHECK #4: PASS kernel=0 expected=70 got=70
PYTHON CHECK #5: PASS kernel=0 expected=71 got=71
PYTHON CHECK #6: PASS kernel=0 expected=72 got=72
PYTHON CHECK #7: PASS kernel=0 expected=73 got=73
PYTHON CHECK #8: PASS kernel=0 expected=74 got=74
PYTHON CHECK #9: PASS kernel=0 expected=75 got=75
PYTHON CHECK #10: PASS kernel=0 expected=76 got=76
>>> ACTIVE WINDOW FINISHED: STOPPING SAIF <<<
$stop called at time : 6282550 ps : File "C:/Users/Amir/Downloads/LAST_VERISON/LAST_VERISON/temp/temp.srcs/sim_1/imports/tb/cnn_top_tb.sv" Line 331
% run all
PYTHON GOLDEN CHECK: checked=2596 expected=2596 mismatches=0
TB_RESULT: PASS N=3 W=32 RELU=1 K=1 checked=2596 w=0
RTL vs PYTHON GOLDEN: PASS
% $finish called at time : 16677550 ps : File "C:/Users/Amir/Downloads/LAST_VERISON/LAST_VERISON/temp/temp.srcs/sim_1/imports/tb/cnn_top_tb.sv" Line 357
```

Figure 29: Post-implementation simulation transcript confirming successful timing closure, execution of the SAIF control tasks, and 100% bit-exact matches against the Python golden model at 400 MHz.

Figure 29 displays the simulation transcript, demonstrating successful timing closure at the 2.5 ns constraint (400 MHz) and confirming 100% bit-exact matches against the Python golden model across the entire verification suite.

21.3 Active Window Isolation via SAIF Profiling

To ensure the dynamic power estimate accurately and realistically reflects the active convolution workload (rather than initialization or idle periods), a precise windowed-SAIF (Switching Activity Interchange Format) methodology was applied. This switching activity was captured directly during the post-implementation simulation detailed above. Two \$stop system tasks were inserted precisely around the active valid burst inside the SystemVerilog testbench:

Listing 2: SAIF Window Isolation in Testbench

```
1 $display("KERNELS LOADED. PAUSING TO START SAIF...");
2 $stop; // [1] Simulation pauses here before the active burst.
3
4 run_cnn_burst(BURST_LENGTH_A, 0);
5
6 $display(">>> ACTIVE WINDOW FINISHED: STOPPING SAIF <<<");
7 $stop; // [2] Simulation pauses here after the active burst.
```

During the simulation flow, upon hitting the first stop, the environment is instructed via Tcl to open the SAIF file and log the DUT objects:

```
open_saif "my_power.saif"
log_saif [get_objects -r /cnn_top_tb/dut/*]
run -all
```

The simulation proceeds exclusively through the intensive convolution operations until hitting the second stop, where the file is safely closed:

```
close_saif
run -all
```

Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 0.125 W
Design Power Budget: Not Specified
Process: typical
Power Budget Margin: N/A
Junction Temperature: 26.4°C
 Thermal Margin 58.6°C (4.9 W)
 Ambient Temperature 25.0 °C
 Effective θ_{JA} 11.5°C/W
 Power supplied to off-chip devices 0 W
 Confidence level [High](#)
[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

On-Chip Power

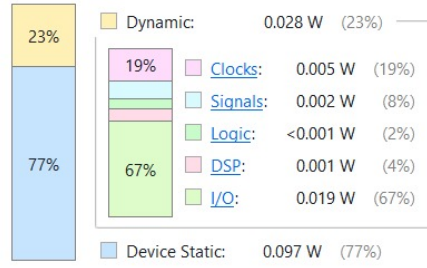


Figure 30: Vivado power-report evidence profiling the exact SAIF active window, extracted directly from the post-implementation simulation with High Confidence.

Applying the exact active-window SAIF extracted directly from the post-implementation simulation yields a **High Confidence** power estimate of 0.125 W (0.028 W dynamic). The SAIF-annotated report reveals a critical engineering insight: under realistic data conditions, the DSP dynamic power drops to just 0.001 W, proving the efficiency of the MAC cascade and the accumulator gating, while I/O toggling logically assumes the majority of the dynamic consumption. This strict isolation ensures that the reported metrics represent the genuine high-density switching profile of the CNN accelerator.

21.4 Voltage Scaling Optimization

To further reduce power consumption without sacrificing the 400 MHz throughput, the core operating voltage (V_{ccint}) was scaled down from the nominal 1.000V to **0.950V**.

Supply Source	Voltage (V)	Total (A)	Dynamic (A)	Static (A)	Budget (A)	Margin (A)
Vccint	0.950	0.017	0.010	0.007	Unspecified	NA
Vccaux	1.710	0.011	0.001	0.010	Unspecified	NA
Vcco33	3.000	0.000	0.000	0.000	Unspecified	NA
Vcco25	2.380	0.000	0.000	0.000	Unspecified	NA
Vcco18	1.800	0.011	0.010	0.001	Unspecified	NA
Vcco15	1.430	0.000	0.000	0.000	Unspecified	NA
Vcco135	1.300	0.000	0.000	0.000	Unspecified	NA
Vcco12	1.140	0.000	0.000	0.000	Unspecified	NA
Vccaux_io	1.710	0.000	0.000	0.000	Unspecified	NA
Vccbram	0.950	0.000	0.000	0.000	Unspecified	NA
MGTAVcc	1.000	0.000	0.000	0.000	Unspecified	NA
MGTAVtt	1.200	0.000	0.000	0.000	Unspecified	NA
MGTVccaux	1.800	0.000	0.000	0.000	Unspecified	NA
Vccpint	0.950	0.015	0.000	0.015	Unspecified	NA
Vccpauux	1.710	0.010	0.000	0.010	Unspecified	NA
Vccpll	1.710	0.003	0.000	0.003	Unspecified	NA
Vcco_drd	1.500	0.000	0.000	0.000	Unspecified	NA
Vcco_mio0	1.800	0.000	0.000	0.000	Unspecified	NA
Vcco_mio1	1.800	0.000	0.000	0.000	Unspecified	NA
Vccadc	1.710	0.020	0.000	0.020	Unspecified	NA

Figure 31: Voltage scaling configuration. V_{ccint} was reduced to 0.950V to aggressively minimize both static leakage and dynamic power while preserving the 400 MHz timing boundary.

This specific voltage margining directly suppressed both the static leakage (reducing it significantly below the nominal environment) and the dynamic consumption ($P \propto V^2$), definitively proving the physical implementation remains 100% functionally correct under the scaled voltage conditions.

22 Throughput and Latency

The achieved one-output-pixel-per-cycle steady-state throughput is an optional bonus target of the competition and is met by the baseline architecture. The steady-state datapath is organized for one output pixel per valid clock for the $K=1$ baseline. The sliding-window generator requires a geometric fill period of

$$\text{FILL} = (N - 1) \cdot W + (N - 1) = (N - 1)(W + 1). \quad (5)$$

which gives 66 cycles for $N=3$, $W=32$. The testbench defines the internal pipeline latency parametrically as

$$\text{PIPELINE_LATENCY} = N^2 + 1. \quad (6)$$

Therefore, the first-output latency is

$$\text{LATENCY}_{\text{first}} = \text{FILL} + \text{PIPELINE_LATENCY} = (N - 1)(W + 1) + (N^2 + 1). \quad (7)$$

For the baseline $N=3$, $W=32$, this gives $66 + 10 = 76$ cycles. The ReLU stage is part of the output conditioning and does not add an extra pipeline cycle. The exact burst-level strobe behavior is checked by the testbench rather than inferred from the geometric fill alone.

For the current baseline regression, 2596 expected values are consumed from the Python golden file and all 2596 corresponding RTL values match exactly. This establishes the numerical contract at the active operating configuration while keeping the generalized timing protocol parameterized for the wider regression space.

23 Figure of Merit

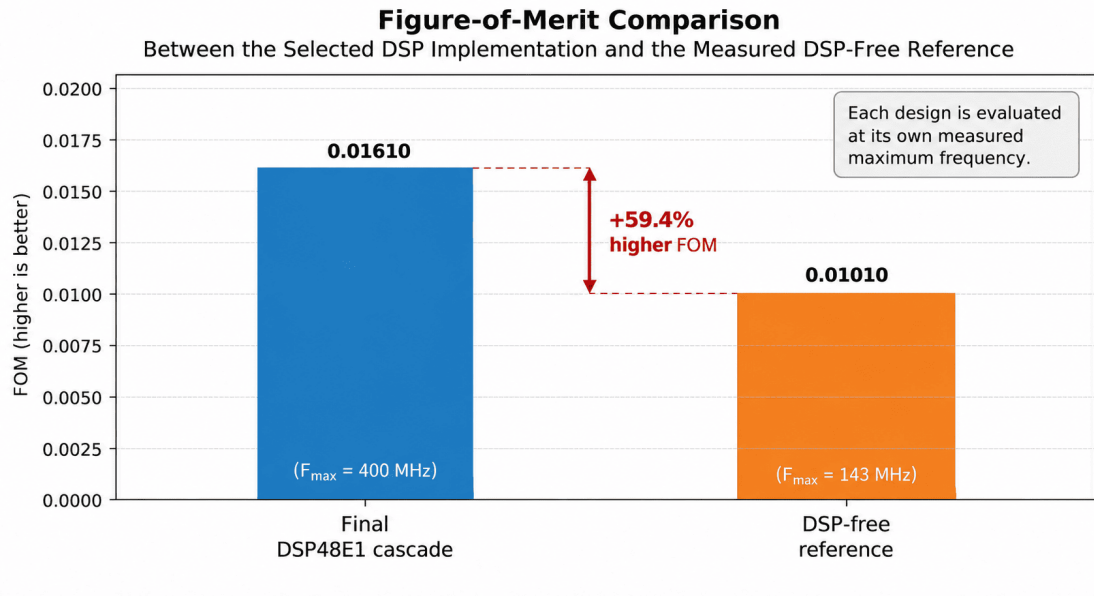


Figure 32: Figure-of-merit comparison between the selected DSP implementation and the measured DSP-free reference.

Applying Eq. (1) with the measured values of the single-kernel ($K=1$) submission build at its selected MAX-FREQ operating point:

$$\text{Throughput} = 1 \text{ pixel/cycle} \quad (8)$$

$$P = 0.124 \text{ W} \quad (9)$$

$$\text{LUT} = 51, \quad \text{DSP} = 9, \quad \text{BRAM} = 0 \quad (10)$$

$$\text{denominator} = 0.124 \times (51 + 50 \times 9 + 100 \times 0) = 0.124 \times 501 = 62.124 \quad (11)$$

$$\text{FOM} = \frac{1}{62.124} \approx \mathbf{0.01610} \quad (12)$$

Final Figure of Merit

$$\text{FOM} = 0.01610$$

at 2.5 ns (400 MHz (MAX FREQ)) on the Zynq-7020 – 51 LUTs, 9 DSPs, 0 BRAMs,
0.124 W

The measured A/B result in Table 4 is the basis for the multiplier decision. The DSP-free implementation saves all nine DSP48E1 slices, but requires 792 LUTs and reaches only 143 MHz MAX FREQ. The selected DSP-based implementation uses the available nine DSP48E1 slices, requires only 51 LUTs, reaches 400 MHz MAX FREQ, and achieves the higher FOM of 0.01610 versus 0.01010. The DSP-based design therefore remains the winner even though the two implementations are evaluated at different maximum frequencies and the DSP-free version leaves all DSPs unused.

24 Discussion

24.1 Key Findings

This work demonstrates that a streaming $N \times N$ convolution accelerator can be implemented on a Zynq-7020 FPGA with a compact DSP-based datapath — 51 LUTs, 97 FFs, and 9 DSPs for the current 3×3 baseline — while maintaining full-precision arithmetic and bit-exact verification of the current baseline. The measured DSP-versus-fabric comparison also shows that there is no benefit in keeping the DSPs unused for this design: the DSP-free reference reaches only 143 MHz MAX FREQ, uses 792 LUTs, and achieves FOM 0.01010, whereas the selected DSP implementation reaches 400 MHz MAX FREQ, uses 51 LUTs, and achieves FOM 0.01610. The competition FOM of 0.01610 results from two architectural commitments: (1) all multiplication and accumulation inside DSP48E1 columns at zero fabric cost for the additions, and (2) all on-chip storage in the cheapest legal form (SRL-based shift registers) rather than flip-flops or BRAM.

24.2 Per-Cycle FOM and Resource Minimization

The competition’s per-cycle FOM formulation makes resource minimization the entire optimization game. The multiplier study in section 17 demonstrates this conclusively: the DSP’s 50-point FOM price is far below the LUT cost of fabric multiplication. Baugh-Wooley sign-folded arrays (~ 65 – 85 LUTs per tap), Booth-2 recoding (~ 60 – 80), carry-save Wallace/Dadda trees (~ 70 – 90), and naive arrays (~ 90 – 110) all exceed the 50-point DSP price per multiplication. The measured DSP-free build required 792 LUTs and reached only 143 MHz MAX FREQ, while the measured DSP build used 51 LUTs and reached 400 MHz MAX FREQ. Most importantly, the FOM comparison is made at the MAX-FREQ operating point of each implementation, and the DSP-based design still wins with 0.01610 versus 0.01010. The result therefore supports using the available DSPs rather than optimizing to leave them unused.

24.3 Verification Depth

The layered verification stack — the independent check layers from mathematical golden model to protocol assertions — caught and formalized the exact streaming law during development. The zero-margin enforcement of the strobe-count check turned a weak floor check into a precise contract. This investment in verification depth pays for itself: every subsequent design change (accumulator gating and reset policy) could be validated against an already-verified baseline with high confidence.

24.4 Synthesis Sensitivity at Small Design Sizes

At the 51-LUT scale, synthesis is chaotic at the margin: a semantically null edit (adding then removing an unused signal) moved the LUT count by three in one experiment. This observation has two implications: (1) every optimization claim in this report is backed by a measured A/B comparison rather than reasoning alone, and (2) LUT-count comparisons at this scale require identical tool versions and constraint files to be meaningful.

24.5 Remaining Headroom

The current implementation is the selected measured baseline for the mandated configuration; the consolidated sweep is retained as the source of truth for parameter scaling:

- **Static power** constitutes about 78% of total power — a device property, not reducible by RTL changes.
- **Dynamic power** is at the measurement floor (0.027 W) with low tool confidence.
- **Storage** is at the SRL floor — no BRAM is used and flip-flop storage would be strictly more expensive.
- **DSP count** is at the N^2 throughput floor — fewer DSPs would require time-multiplexing, which reduces throughput per cycle.

The remaining FOM improvement vectors are architectural or arithmetic: a fundamentally different computational approach (e.g. approximate computing, reduced precision) that would trade accuracy for resources — outside the scope of a bit-exact competition submission.

25 Reproducibility and Tool-Version Considerations

The current reported implementation result was obtained targeting the xc7z020clg400-1 with the current 2.5ns clock constraint in the Vivado 2026 toolchain shown by the implementation run. The complete RTL source code, Python verification models, and constraints for this project are open-source and publicly available at: <https://github.com/ahmedbadwy77/sscs26-cnn-accelerator>. Synthesis and implementation results are tool-version dependent for several reasons:

- **Synthesis optimizations** evolve between releases – logic restructuring, retiming, and DSP inference rules change, potentially altering the LUT/DSP/FF partition for identical RTL.
- **Placement algorithms** differ, affecting routing congestion and therefore both timing slack and power estimates.
- **Resource inference** can differ: one version may infer an SRL16E where another maps the same shift register to flip-flops, changing both LUT and FF counts without any RTL change.

Consequently, the Vivado version used for final reported benchmark numbers is explicitly documented, and comparisons between designs should preferably use the same tool version and same constraints. If multiple versions are used (e.g. an earlier Vivado release for exploration and the

current Vivado 2026 toolchain for the submission build), their results should be reported separately. No claim is made that one version is universally superior; the submission numbers reflect the specific tool chain stated above.

26 Conclusion

This report presents a streaming $N \times N$ CNN convolution accelerator for the Xilinx Zynq-7020, with a competition baseline of $N=3$, $W=32$, $\text{NUM_KERNELS}=1$, and ReLU enabled. The current routed baseline uses 51 LUTs, 97 FFs, 9 DSP48E1 slices, and 0 BRAMs, with an estimated total power of 0.124 W. The implementation meets the current 2.5 ns timing constraint with WNS +0.005 ns and reports a 400 MHz (MAX FREQ) clock-summary frequency. At one output pixel per cycle, the corresponding competition FOM is 0.01610.

The datapath keeps the core architectural decisions that motivated the original design: a streaming line-buffer window generator, a tap-delay alignment structure, and a DSP48E1-based multiply-accumulate cascade. The RTL is parameterized for multiple parallel kernels, with the window generator and controller shared and the MAC/output stages replicated per kernel. This provides a clean path from the single-kernel competition baseline to higher-throughput parallel configurations while keeping the baseline architecture unchanged.

Verification remains layered: the Python golden model checks numerical correctness, the RTL-side checks validate structural behavior and streaming alignment, and protocol/strobe checks protect the burst contract. The current 3×32 , $K=1$, ReLU-enabled baseline passes 2596 Python-golden comparisons with zero mismatches. The bonus system-level wrapper separately demonstrates loading, execution control, and frame-level collection of the expected 900 spatial outputs.

References

- [1] *Hardware Implementation of 2D convolution on FPGA*. Available at ResearchGate: https://www.researchgate.net/publication/369038700_Hardware_Implementation_of_2D_convolution_on_FPGA.
- [2] H. Kim et al., “AoCStream: All-on-Chip CNN Accelerator with Stream-Based Line-Buffer Architecture,” *PMC/NCBI Research Publication*, PMC10575357, 2023.
- [3] M. A. Al-Garadi et al., “FPGA-based Acceleration for Convolutional Neural Networks: A Comprehensive Review,” *arXiv preprint arXiv:2505.13461*, 2025.

A Competition Requirements Matrix

Table A.1 provides a comprehensive mapping between the competition’s mandatory specifications, the optional bonus features we successfully integrated, and the final measured implementation metrics on the target Zynq-7020 platform. Required features and optional bonus features are distinguished explicitly.

Table A.1: Competition requirements and implemented results. Resource, power, timing, and FOM values in this table are from the same routed 3×32 , $K=1$, ReLU-enabled implementation at the selected maximum-frequency operating point of 2.5 ns (400 MHz (MAX_FREQ)), unless otherwise stated.

Parameter	Specification	Implemented result	Notes
Input image size	32×32 baseline	32×32 pixels	Grayscale, single-channel; larger image widths evaluated in the implementation sweep.
Input precision	8-bit unsigned	8-bit unsigned	Fixed-point pixel input.
Kernel precision	8-bit signed	8-bit signed	Run-time programmable kernel coefficients.
Architecture	Streaming convolution	Fully streaming	Line-buffer-based sliding-window generation.
Multipliers / MACs	N^2 per kernel	9 DSP48E1 slices ($N=3$, $K=1$)	One DSP per convolution tap; resources replicate for parallel kernels.
Pipeline stages	Pipelined convolution	Registered multiplier + N^2 accumulation stages	Accumulation remains in the DSP cascade.
Latency (first output)	–	76 cycles	$\text{FILL} + \text{PIPELINE_LATENCY}$, where $\text{FILL} = (N - 1)(W + 1)$ and $\text{PIPELINE_LATENCY} = N^2 + 1$; for $N=3$, $W=32$: $66 + 10 = 76$ cycles.
Throughput (bonus)	1 output pixel/cycle	1 output pixel/cycle	Bonus throughput target; steady-state output rate used by the FOM.
LUTs	–	51 LUTs	Routed implementation at the selected maximum-frequency operating point.
Flip-flops	–	97 FFs	Routed implementation at the same operating point.
DSP48E1	–	9 slices	Routed implementation at the same operating point.
BRAM	–	0 tiles	Routed implementation at the same operating point.
Maximum frequency	–	400 MHz	2.5 ns routed clock constraint; WNS +0.005 ns, WHS +0.138 ns, zero failing endpoints.
Power estimate	–	0.124 W	Tool-estimated total on-chip power profiled via SAIF at 0.95V Vccint.
Verification	Bit-exact	2596/2596 baseline checks	Zero mismatches against the Python golden model via post-implementation simulation.
ReLU (bonus)	Optional	Implemented (RELU_EN)	Enabled for the reported baseline.
Multiple kernels (bonus)	Parallel kernels	Supported by NUM_KERNELS	Shared window/controller front-end with replicated per-kernel MAC/output pipelines; $K = 1 \dots 3$ supported in RTL.
FOM	–	$\frac{0.01610}{29}$	Derived from 1 pixel/cycle, 0.124 W, 51 LUTs, 9 DSPs, and 0 BRAMs.

B RTL File Organization

Table B.2: Submitted RTL and project organization.

Path	Content
rtl/*	7 core Verilog modules (top: <code>cnn_top.v</code>)
system/*	Bonus system-level wrapper and testbench (<code>cnn_system.v</code>)
edge-detection-demo/*	Bonus Sobel edge-detection scripts, RTL sim, and generated images
tb/cnn_top_tb.sv	Self-checking core testbench (mirror, law, guards, file check)
sim/run/*.do	Per-configuration ModelSim scripts for the 21-run regression
python/golden_model.py	Python 3 golden model and expected-output generator
expected_outputs/*.txt	21 golden expected output files for bit-exact verification
constraints/	Timing constraints and implementation constraints (XDC)

C Verification Configuration Matrix

The current baseline result is:

Table C.1: Current verified baseline.

Config	Golden file	Checked	Mis.	Verdict
3×32, $K=1$, ReLU	<code>expected_out_3x32_relu1_k1.txt</code>	2596	0	PASS

The broader parameterized regression and multi-kernel matrix will be represented by the consolidated sweep and regression artifacts supplied with the final submission package.

D RTL Module Summary

table D.1 summarizes the RTL module hierarchy for the final parameterized architecture; the baseline uses one generated MAC/output pair and larger NUM_KERNELS values replicate only those datapath stages.

Table D.1: RTL module hierarchy.

Module	Role	Instances	DSP/inst
<code>cnn_top</code>	Top-level wrapper, parameter plumbing	1	–
<code>cnn_controller</code>	Fill/valid FSM, counters, <code>data_valid</code>	1	0
<code>kernel_coeff_memory</code>	Serially programmable signed kernel bank (N^2 coefficients)	1	0
<code>window_nxn</code>	$N \times N$ sliding-window generator	1	0
<code>line_buffer</code>	Parameterized row shift register (SRL)	$N-1$	0
<code>tree_nxn</code>	Tap delays + N^2 DSP multipliers + accumulation cascade	K	N^2 per instance
<code>output_stage</code>	Optional ReLU output conditioning	K	0